



github.com/sailuh

Kaiaulu Config, Executable, Unit Test, and Data Schema Refactoring

Cohen Ruport, Gi Young (Michelle) Back, Colby Fuke, John Flinn
Sponsors: Carlos Paradis and Rick Kazman
ICS 496 Capstone Project - Spring 2026



What is Kaiaulu?

Kaiaulu is an R package and common interface that helps developers in mining software repositories to better understand the evolving software development communities and the artifacts they produce.

Objectives

- Refactor Kaiaulu's configurations into reusable project and analysis .yaml files.
- Refactor/create executables which provide command line access to the functions which Kaiaulu has.
- Refactor/create new unit tests as well as contribute to the already existing stability measures in place
- Create a MySQL Workbench diagram to document table and module connections, along with a guide for future contributors to follow

Methodology

Project Management

- Both weekly group meetings internally as well as with our sponsor
- Documentation with GitHub Issues + PRs + Kanban Project Manager

Group Roles

- Project Manager:** Cohen
- Liaison:** Michelle
- Quality Manager:** Colby
- Code Architect:** John

Configurations (.yaml) : : CHEAT SHEET



About

Kaiaulu uses configuration files to store data parameters that are used to run analyses. Instead of having these values in notebooks or scripts, these configuration files let users reuse the same workflows across different projects and analyses.

By splitting the configurations into **project.yaml** (1) and **analysis.yaml** (2) files, it reduces the duplication of data parameters and allows users to mix and match different project and analysis configs.

Project Configuration

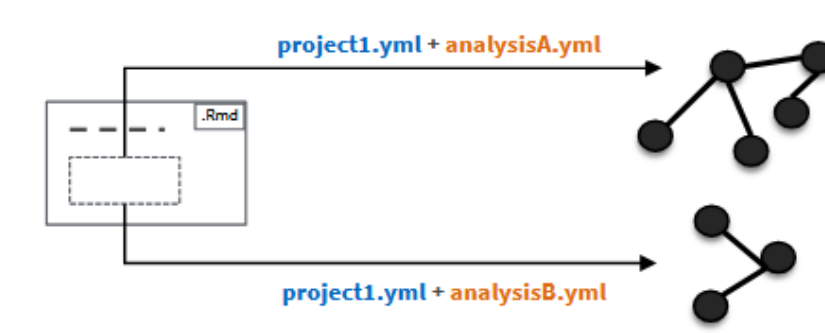
In the configuration, **project.yaml** files (1) define project-related data such as version control, issue trackers, and mailing lists.



These files are specific to a project and remain consistent across different analyses.

Analysis Configuration

Users can define in the **analysis.yaml** (2) configuration files filters heuristics and tool parameters (e.g. depends, uctags).

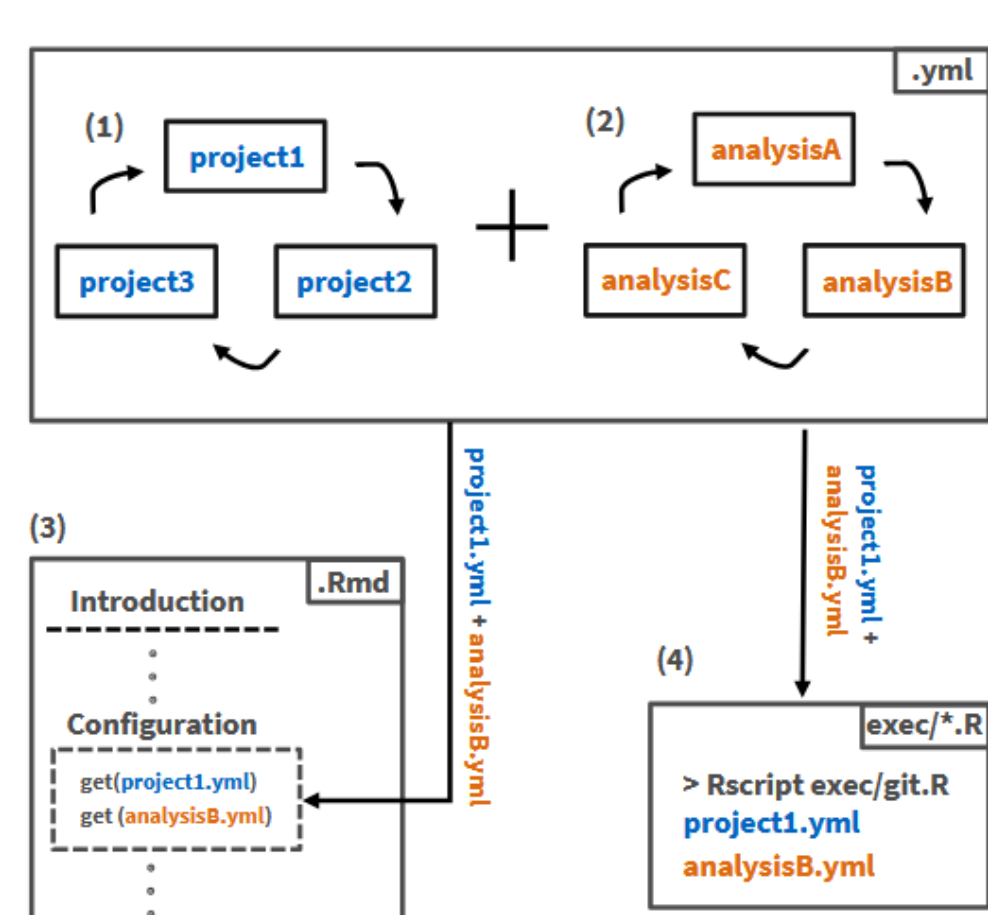


These files can be reused across different projects and define how data is analyzed.

Kaiaulu

Interchangeable Configurations

Project.yaml files (1) are combined with any **analysis.yaml** files (2). This allows users to run the same analysis on multiple projects or run different analyses on the same project, reducing duplication of configuration sections. The combined configuration can be used both in notebooks and scripts.



In the notebooks (3), configuration files can be found near the beginning of the file. The notebook parses **project.yaml** (1) and **analysis.yaml** (2) files and gets the values of the parameters with get functions. This allows users to modify parameters in the .yaml files instead of the notebooks.

In the executables (4), both **project.yaml** (1) and **analysis.yaml** (2) files are passed directly as arguments through the command line. The script runs the selected configuration without modification. Although using executables makes **project.yaml** (1) and **analysis.yaml** (2) files fixed during execution, the advantages of using executables versus notebooks are that they can run on server side and do not require knowledge of R.

CC BY SA Gi Young (Michelle) Back, Carlos Paradis • Kaiaulu package version 0.0.0.9700 (in development) • Updated: 2026-04

Config API

Kaiaulu uses get functions to extract values from configuration files. These functions allow notebooks and executables to access parameters in **project.yaml** (1) and **analysis.yaml** (2) files.

Project

The following APIs are used to retrieve parameters from the **project.yaml** configurations (1).

- get_git_repo_path()**
Returns the local path of the project's Git repository.
- get_jira_issues_path()**
Retrieves the local path to stored JIRA issue data.

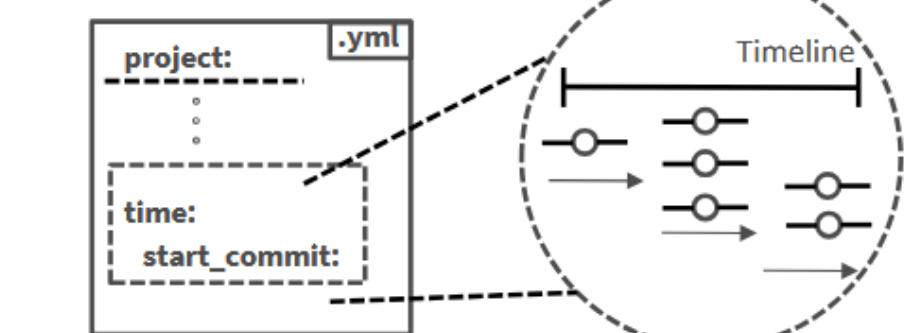
Analysis

The following APIs are used to retrieve parameters from the **analysis.yaml** configurations (2).

- get_file_extension()**
Returns file types (e.g. c, java) used to filter source files.
- get_depends_keep_dependencies()**
Retrieves dependency types specified in analysis configuration.

Temporal Analysis

Temporal parameters in **analysis.yaml** files (2) allow users to analyze a project over time. This allows users to compare data from different timestamps and observe changes and activity in code structure over time.



Kaiaulu Executables: : CHEAT SHEET



About

Executables provide access to Kaiaulu's functions through a command line interface. This allows other tools to build on Kaiaulu capabilities or use it server-side.

Project Config Setup

The first part of running any vignette is setting up your project configuration file (examples in conf/kaiaulu).

Required Fields

- mailing_list: ...
- issue_tracker: ...

Analysis Config Setup

An analysis file is also required to specify filters and parameters

Required Fields

- filters: ...
- analysis: ...

Tools Config Setup

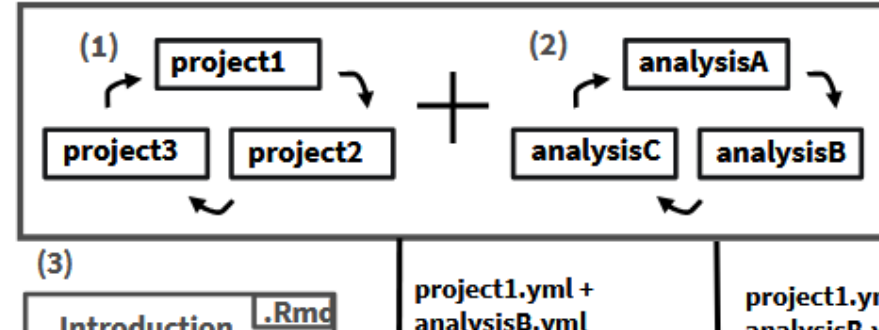
The required external tools must be specified in tools.yaml:

Required Fields

- perceval: ...

Exec Configuration

Executables require configuration files to find information about the project, analysis parameters and tools needed. The project and analysis config are also used by notebooks in Kaiaulu.



Kaiaulu executables provide data from sources such as Mail, JIRA, and GitHub.

Kaiaulu

Interchangeable Configurations

Any **project.yaml** files (1) can be combined with any **analysis.yaml** files (2). This allows users to run the same analysis on multiple projects or run different analyses on the same project, reducing duplication of configuration sections. See Configuration (.yaml) cheatsheet for more information.

Sailuh tools such as pykumu rely on kaiaulu executables to download and refresh data. Kaiaulu executables are also made to provide kaiaulu's functions to users, tools, and projects outside of the sailuh ecosystem.

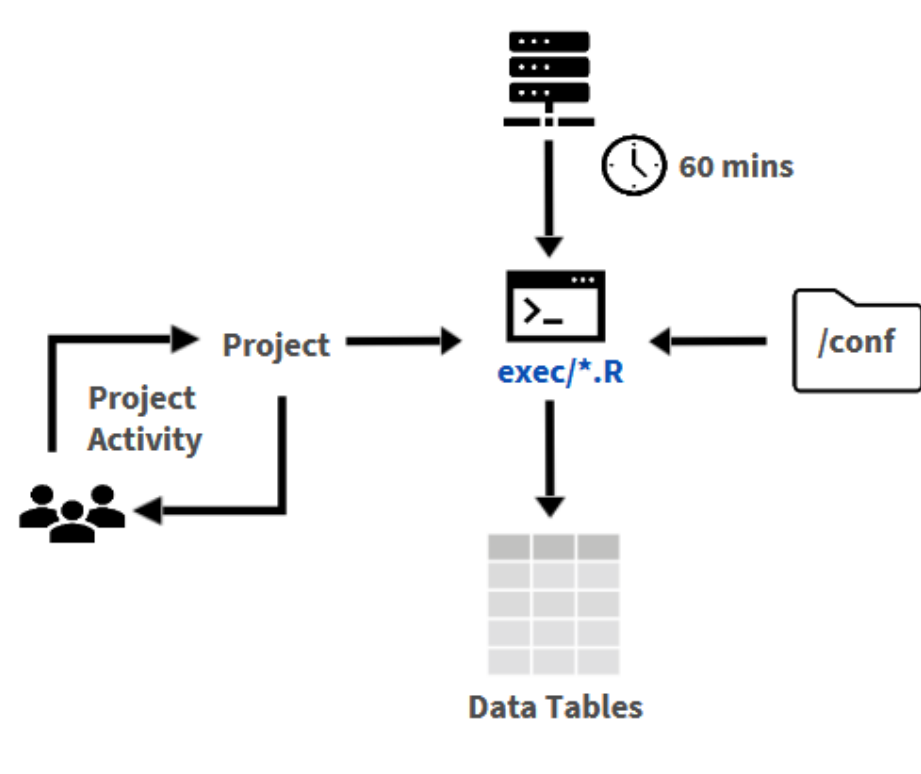


Kaiaulu executables provide data from sources such as Mail, JIRA, and GitHub.

Kaiaulu

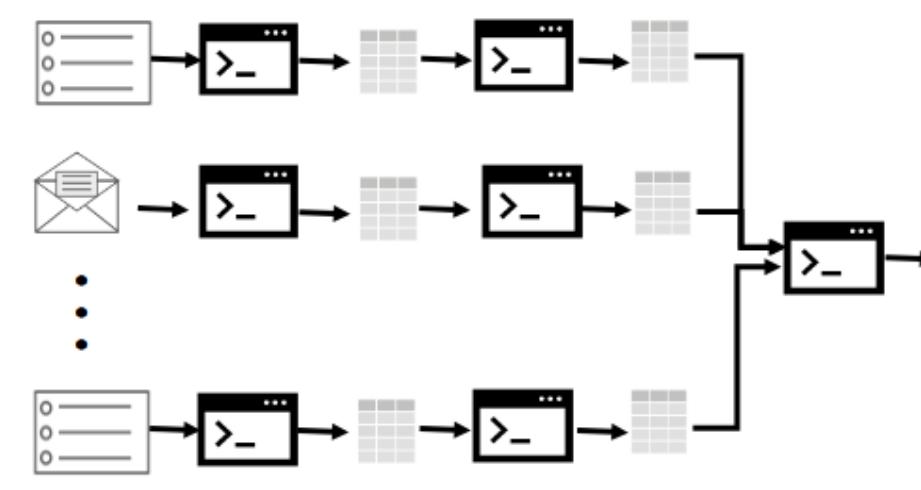
Server Side Execution

Cron jobs can be used to call refresh scripts every x minutes (E.g., 60 minutes). The refresher script is called regularly to ensure that local data is always up to date without manual intervention.



Data Pipelines

Executables remove the need to manually write R scripts, fetch data, or manage complex workflows. Instead of handling APIs and parsing data manually, Kaiaulu executables do this using configuration files. Executables can also be chained together to construct more complex pipelines.



CC BY SA Cohen Ruport, Carlos Paradis • Kaiaulu package version 0.0.0.9700 (in development) • Updated: 2026-04

Unit Tests : : CHEAT SHEET



About

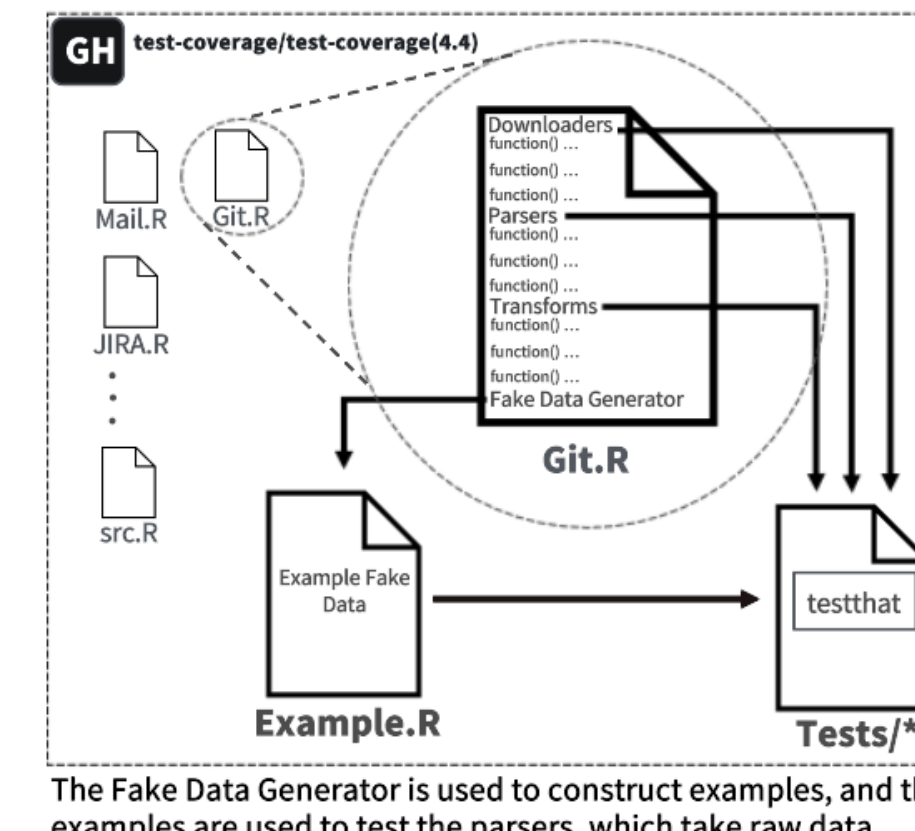
Kaiaulu has multiple measures in place to ensure that its codebase remains maintainable. Try catch statements and tests let functions fail gracefully and ensure functions work as intended. Codecov and GitHub Actions run the tests as well as other checks for every commit to a Pull Request and will merge if any tests fail or code coverage conditions are not met.

Failing Gracefully: Try Catch

Try catch statements allow errors that propagate through files to be caught gracefully.

A common example in Kaiaulu is an invalid Git Log Path specified in a parser, which moves to a transform, and eventually throws an error in an .Rmd file.

Test Framework



Kaiaulu

Examples

Kaiaulu's Example.R is used to create fake data examples. These mimic real raw data cases.

Fake Data Generator

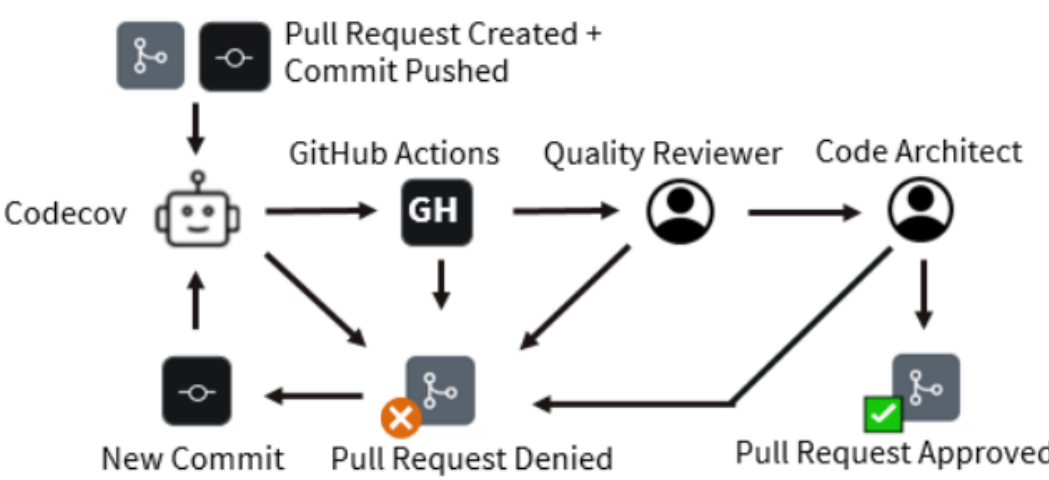
Fake Data Generator functions are at the bottom of each R file. They are called in Example.R to create the fake data cases.

Tests

Tests for Kaiaulu are made using R's testthat framework. Fake examples are used for parser tests, which parse fake data.

Code Review Workflow

The Kaiaulu code review workflow follows a multiple step process.



Codecov will deny a commit based on code test coverage.

GitHub Actions

GitHub Actions will deny a commit if any tests from the testthat folder fail.

Quality Reviewer

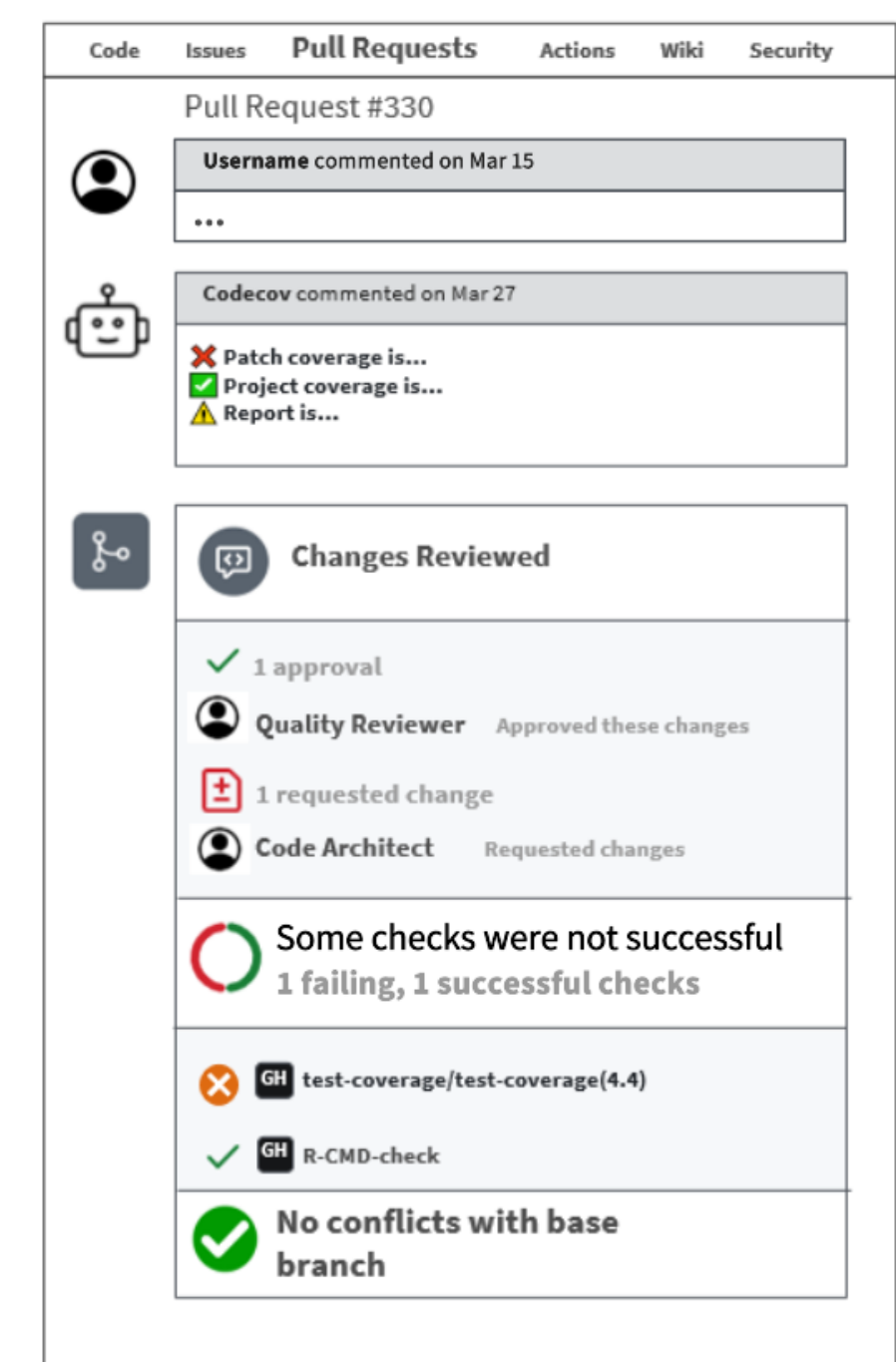
The Quality Reviewer is a user role that runs the committed code on their own machine, ensuring that code is not reliant on the hosts' personal setup.

Code Architect

The Code Architect is a user role that checks to see that the committed code matches the conventions of other similar files in Kaiaulu.

Pull Request Workflow

In practice, the workflow process happens in the Pull Request. Codecov has a bot comment on the request and report its patch and project coverage, as well as deny the PR if it deems them too low. The users assigned to the project's Quality Reviewer and Code Architect can then either approve or request changes to the commit files. GitHub Actions can inform the user specifically of which tests failed when running the commit code.



CC BY SA Colby Fuke, Carlos Paradis • Kaiaulu package version 0.0.0.9700 (in development) • Updated: 2026-04

Data Schema: : CHEAT SHEET



About

Kaiaulu supports combining of multiple sources, such as Git, Jira, and GitHub for analysis. This cheat sheet showcases the data schema for Kaiaulu that facilitates identifying how to combine data sources of interest to the user.

Layers

Data Layer

Kaiaulu Data Layer represents the various types of raw data associated to different software development infrastructure (Mail, JIRA, GitHub). This dataset is generally difficult to connect to other data sources or is not easily available in table form for data exploration.

Schema Layer

In Kaiaulu, raw data sources are represented as tables. This facilitates both data exploration and connect tables containing common columns. Kaiaulu standardizes the nomenclature of columns from different data sources, and the data schema facilitates identifying what columns can be combined for different analysis (e.g. bug_count.Rmd).

Module Layer

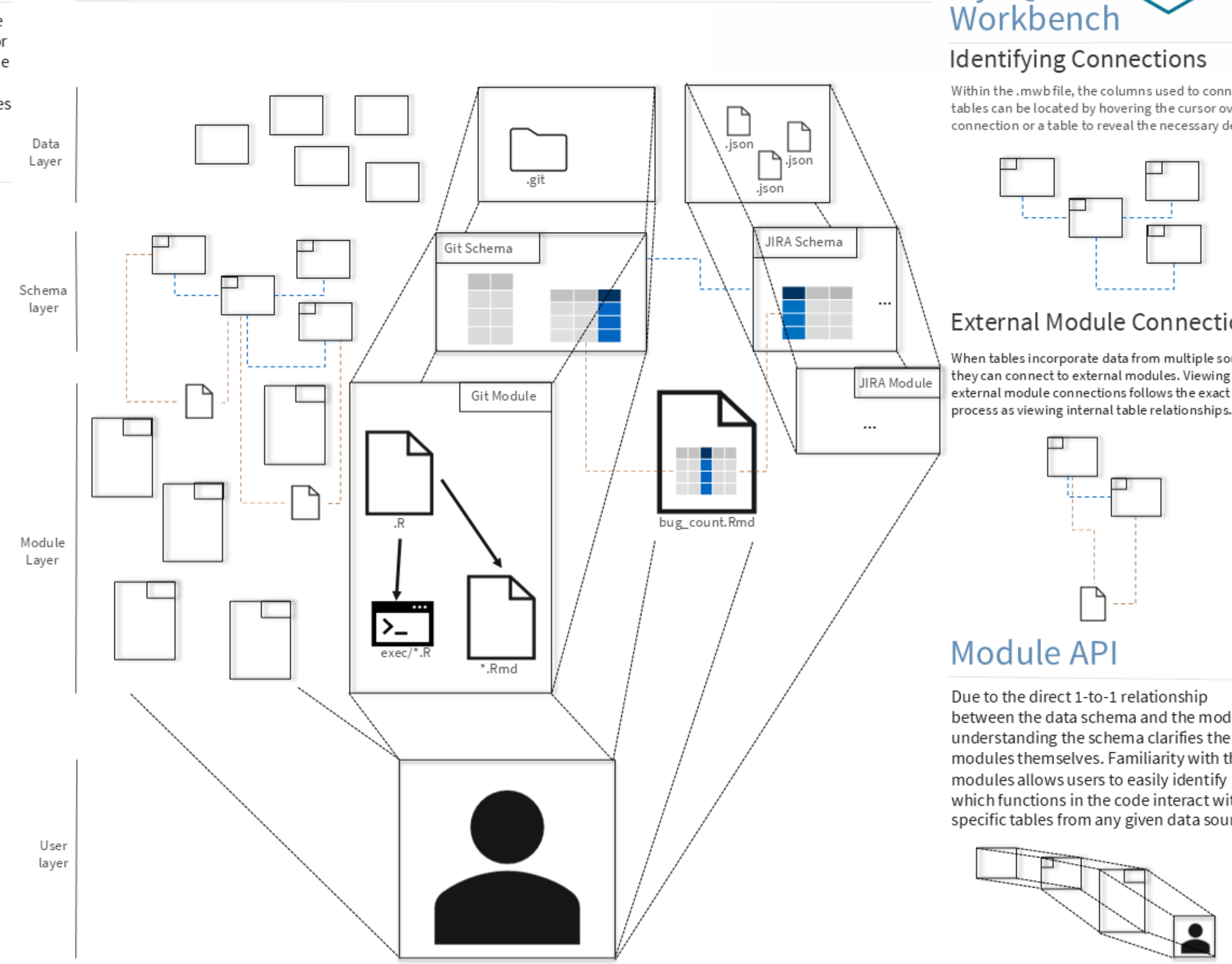
In Kaiaulu, files are traditionally separated by their file type, such as functions (.R), documentation (.Rd), and Notebooks (.Rmd). However, the reference API site and MySQL Workbench structure information by grouping it based on the data source. The benefit of this approach is to simplify the process by limiting the scope of files needed for a given analysis.

User Layer

The User Layer provides tools, like Notebooks and exec scripts, that act as accessible entry points to the tools within the Module Layer. This connection lets users easily run analyses without needing to navigate the complex underlying code.

Kaiaulu

Kaiaulu Architecture



CC BY SA John Flinn, Carlos Paradis • Kaiaulu package version 0.0.0.9700 (in development) • Updated: 2026-04

Tech Used

Developer Tools:

Rstudio, MySQL Workbench, GitHub, Codecov, Git
Languages:
R, Python
Third-Party Tools:
Perceval, Depends, Scitools, Understand, Jira, SCC, DV8

Challenges

- Clearly defining the scope and detail of our refactoring work
- Becoming familiar with how Kaiaulu was built and documented
- All team members were unfamiliar with the R language
- Ensuring clear communication and collaboration between all team members

Tasks Accomplished

- Config Refactoring:** Separated configurations into project and analysis .yaml files, enabling users interchangeable use across notebooks and execs.
- Executable Refactoring:** Standardized executables and created new executable files which provide users access to Kaiaulu's functions through a command line interface.
- Unit Test Refactoring:** Proposed and created new unit tests, fake data example functions, try catch statements, and created a new type of test to test the third-party tools that Kaiaulu uses.
- Data Schema Refactoring:** Developed and implemented a documentation system for module tables and their corresponding merges.
- Cheatsheet Creation:** Each member of the group created a cheatsheet that encapsulates the entirety of our work, allowing users to quickly become familiar our contributions.

Learnings & Conclusion

- We learned how to more effectively work in a group setting with clearly defined roles and responsibilities, as well as assisting each other when necessary
- We gained project management experience by using multiple different scheduling methods
- Iterating with our sponsor gave us invaluable experience with professional communication
- By the end of the semester, we were familiar with R, RStudio, and Python notebooks, and the process of how quality code makes its way into an open-source project